

Bipartite Graph & Flow

lanos212

网络流

- 网络流概念
- 最大流
- 最小割
- 费用流
- 上下界网络流

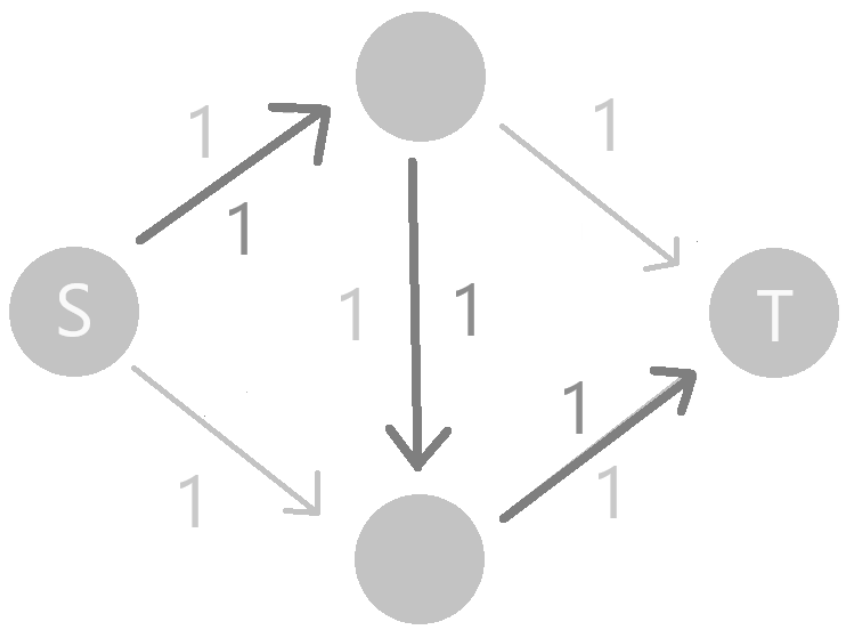
网络流概念

<https://oi-wiki.org/graph/flow/max-flow/>

最大流

最大流问题，顾名思义，需要求出某网络的一个流 f ，使得 $|f|$ 最大。

我们会想到网络流是否像匈牙利算法一样直接搜索以不断增广：



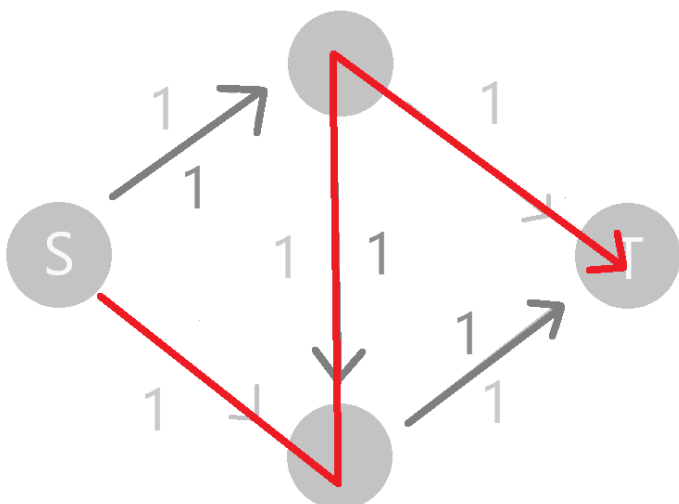
然而在上面这张图中，最大流是 2，直接搜不一定可以得到最大流。

最大流算法的核心思想在于，对每条边建一条与其容量互补的反向边。

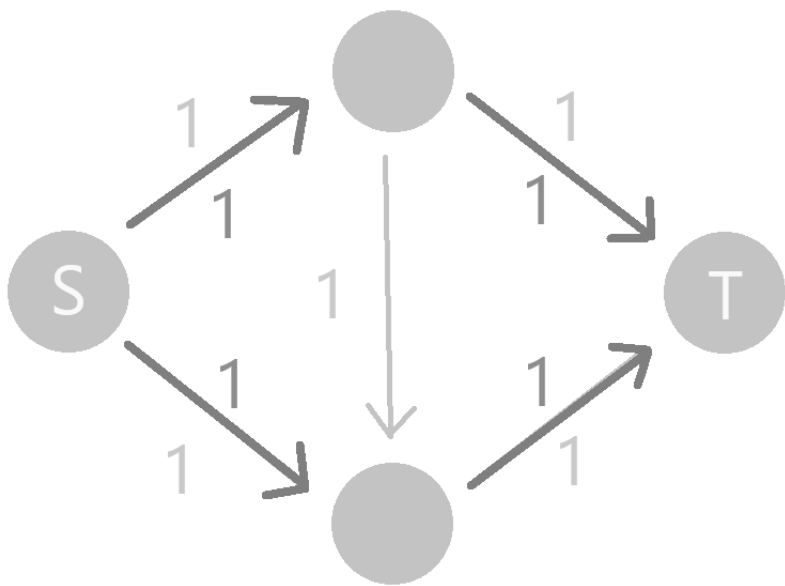
这里我们对于一条边 (u, v) ，约定 $f(u, v) = -f(v, u)$ 。

这相当于多了一个反悔的操作，当我们流过正向边时，正向边减去对应的容量，反向边就增加对应的容量；当我们流过反向边时，相当于将原本的某部分流改道。

比如我们对上面的情况加上反向边继续增广：



我们就可以得到一个容量更大的流：



实际上，当我们增加上反向边，这样做就是正确的。

下面给出证明。

最大流最小割定理

- 对于任意网络 $G = (V, E)$, 其上的最大流 f 和最小割 $\{S, T\}$ 总是满足 $|f| = ||S, T||$ 。

为了证明该定理, 我们先证明一个引理, 其可以通过简单推式证明。

- 对于网络 $G = (V, E)$, 任取一个流 f 和一个割 $\{S, T\}$, 总是有 f 的流量 $\leq$ 割 $\{S, T\}$ 的容量, 等号成立当且仅当所有 S 连到 T 的边满流, T 连到 S 的边空流。

推导式子: <https://oi-wiki.org/graph/flow/max-flow/>

上面的做法在找不到增广路时停止，考虑分析此时流 f 的情况。

设在残余网络 G_f 上，原点可以到达的点集为 S ，又设 $T = V \setminus S$ 。

显然 S, T 是原网络的一个割，我们考虑所有满足 $u \in S, v \in T$ 的边 (u, v) ，有两种情况（下文中 E 为原网络的边集合）：

- $(u, v) \in E$ ，此时剩余流量为 0，即 $c(u, v) = f(u, v)$ ，可以得到这些边皆满流。
- $(v, u) \in E$ ，此时剩余流量为 0，即 $f(u, v) = c(u, v) = 0$ ，有 $f(v, u) = -f(u, v) = 0$ ，可以得到这些 (v, u) 皆空流。

这恰好满足了引理取等条件，做法正确性得到保证。

EK 算法大致流程

定义残余网络 G_f 为网络 G 在找到流 f 后，剩下的边与原本的点组成的网络。

EK 算法大致流程为，每次用 BFS 找一条残余网络上**最短的**增广路，找到增广路上容量最小的一条边，假设其容量为 D ，那么我们对于这条边进行一次流量为 D 的增广。

一直做上述过程，直到找不到增广路为止。

该算法时间复杂度为 $O(|V||E|^2)$ ，下面将给出证明。

(对于时间复杂度部分，我们都假设 $O(|V|) \prec O(|E|)$)

EK 算法时间复杂度证明

首先对于一次增广的时间复杂度为 $O(|E|)$ 。

我们现在只需要证明增广次数的上界是 $O(|V||E|)$ 。

下文中， f 表示当前流， f' 表示进行某次增广后的流。

定义 $d_f(u)$ 表示找到当前流时，残余网络上源点到 u 的最短路长度（长度定义为路径上的边数）， $d_{f'}(u)$ 同理。

引入一个引理：

- 对于任意结点 u ，增广总是使得 $d_{f'}(u) \geq d_f(u)$ 。

考虑反证法，我们找到增广后最短路变小的所有点中，**新的最短路最短**的一个点 u ，那么首先有 $d_{f'}(u) < d_f(u)$ 。

我们在 f' 上，找到源点到 u 最短路上， u 的前一个点 v ，那么存在等式 $d_{f'}(v) = d_{f'}(u) - 1$ 。

那么由于 u 是最短路变小的点中，新的最短路最小的，为了让 v 不抢走 u 最小的地位，一定有 $d_{f'}(v) \geq d_f(v)$ 。

将这三个式子整理起来：

$$\begin{cases} d_{f'}(v) \geq d_f(v) \\ d_{f'}(u) < d_f(u) \\ d_{f'}(v) = d_{f'}(u) - 1 \end{cases}$$

我们可以放缩出 $d_f(v) < d_f(u) - 1$ 。

有了这个式子，我们再对 v 与 u 之间的边进行讨论。

设 E_f 为残余网络 G_f 中的边集, $E_{f'}$ 同理。

- $(v, u) \in E_f$ 时, 根据 BFS 的性质, $d_f(v) \geq d_f(u) - 1$, 矛盾。
- $(v, u) \notin E_f$ 时, 根据 v 的定义, 一定有 $(v, u) \in E_{f'}$, 因此这条边一定是在这次增广中, 流过了 (u, v) 这条边导致的, 所以 $d_f(v) = d_f(u) + 1$, 矛盾。

综上所述, 所有情况都将导出矛盾, 所以假设不成立, 原引理得证。

现在我们就可以来证明增广次数上界了。

我们称一次增广中，增广路上容量最小的边为**饱和边**。

那么当一条边作为饱和边时，它的所有容量都将用完，那么下一次一定是它的反向边先被增广，当它的反向边再作为饱和边时，同理之后先被增广的是正向边，如此交替进行。

那么根据增广后源点到自己的最短路不会减少的引理，这样往返交替增广一次会让源点到当前点的最短路距离至少增加 2。

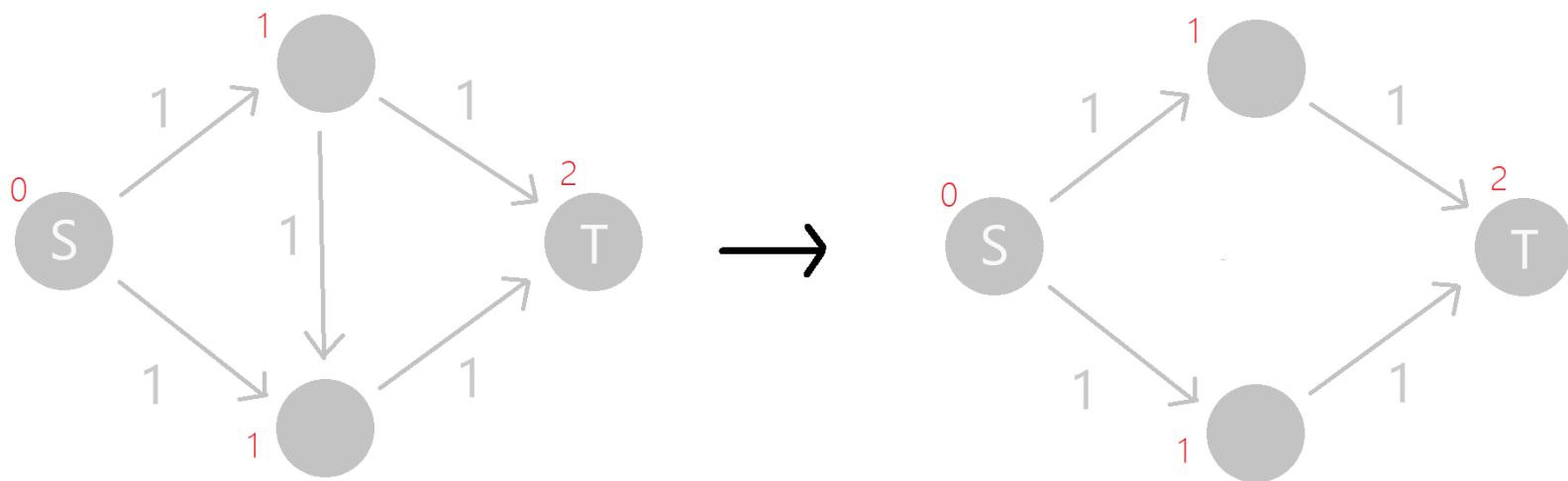
由于最短路距离不会超过 $|V|$ ，所以每条边被当作饱和边的次数是 $O(|V|)$ 级别的，总增广次数即为 $O(|V||E|)$ 级别。
最终可以得到 EK 算法时间复杂度为 $O(|V||E|^2)$ 。

Dinic 算法大致流程

Dinic 算法通过对图分层，以及当前弧优化使其时间复杂度更为优秀。

(多路增广只是起到常数优化作用，它对复杂度没有影响)

对图分层的操作用 BFS 实现，在残余网络上找到源点到每个点的最短路距离，对于一个最短路距离为 d 的点，我们只保留其连向距离为 $d + 1$ 的点的边。



定义 G_L 为我们得到的分层图。

如果我们在分层图上找到一个最大的增广流 f_b ，使得 G_L 上找不到更大的流，我们称 f_b 是 G_L 的阻塞流。

Dinic 算法的大致流程如下：

- 在当前残余网络 G_f 上 BFS 出分层图 G_L 。
- 在分层图 G_L 上 DFS 出阻塞流 f_b （注意当前弧优化！）。
- 将阻塞流并入当前流 f ，即 $f \leftarrow f + f_b$ 。
- 重复上述过程直到不存在源点到汇点的路径，此时 f 为最大流。

时间复杂度证明

Dinic 的时间复杂度为 $O(|V|^2|E|)$ ，并在**大部分图上效率非常优秀**。

由于篇幅问题，完整证明不在这里写出，给出大致思路。

- 可以证明，Dinic 单次 DFS 找阻塞流的时间复杂度为 $O(|V||E|)$ 。
- 可以证明，Dinic 每次的**分层图层数递增**，由于层数不超过 $O(V)$ ，得到我们最多只需找 $O(|V|)$ 次阻塞流。
- 综上所述，Dinic 的时间复杂度为 $O(|V|^2|E|)$ 。

完整证明：<https://oi-wiki.org/graph/flow/max-flow/>

特殊情况时间复杂度分析（单位容量）

在单位容量的网络中，其单次找阻塞流时间复杂度为 $O(|E|)$ 。

证明是简单的，单位容量的网络中，每次增广会让增广路上的边全部消失，每条边只会至多遍历一次，得证。

在单位容量网络中，Dinic 算法增广轮数是 $O(|E|^{\frac{1}{2}})$ 的。

假设当前已经增广了 $|E|^{\frac{1}{2}}$ 轮，由于分层图层数是递增的，所以此时距离源点最远点距离为 $\geq |E|^{\frac{1}{2}}$ 。

我们现在再进行一次分层，但是只对点分层，保留残余网络所有的边。

根据鸽巢原理，当前残余网络一定有两层之间，从距离源点近(layer)连向距离远的(layer)的边数 $\leq \frac{|E|}{|E|^{\frac{1}{2}}} = |E|^{\frac{1}{2}}$ 。

发现这些边取出，为当前残余网络上的边的一组割。

由于最大流最小割定理，当前只需再增广 $O(|E|^{\frac{1}{2}})$ 次，得证。

在单位容量网络中，Dinic 算法增广轮数是 $O(|V|^{\frac{2}{3}})$ 的。

假设当前已经增广了 $2|V|^{\frac{2}{3}}$ 轮，再只对点进行分层。

此时一定存在两个相邻层，使得两层点数都 $\leq |V|^{\frac{1}{3}}$ 。

考虑距离近(layer)连向距离远的(layer)的边，至多有 $|V|^{\frac{2}{3}}$ 条。

发现这些边取出，为当前残余网络上的边的一组割。

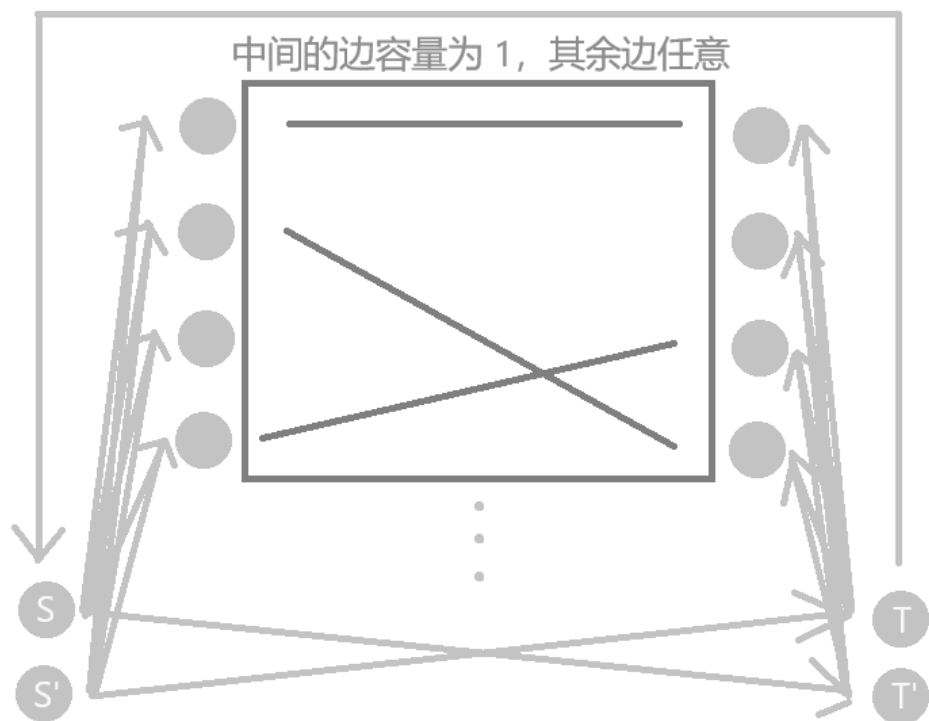
由于最大流最小割定理，当前只需再增广 $|V|^{\frac{2}{3}}$ 次，得证。

(单位容量可视作所有边容量为 1，故边数与割的容量同级)

经过上述证明，我们得到单位容量的网络中，Dinic 算法时间复杂度为 $O(|E| \min(|E|^{\frac{1}{2}}, |V|^{\frac{2}{3}}))$ 。

虽然这个**结论十分重要**，但是推导过程也值得借鉴。

例如我们可以证明 🐶 秒掉 [*3100](#) 所建模的网络时间复杂度同上：



更特殊的网络时间复杂度分析

单位容量网络，且除了源汇点外每个点出度 = 1 **或者** 入度 = 1。

这种网络的最大流一定可以分成若干条**点不交**的最短路。

假设当前已经增广了 $|V|^{\frac{1}{2}}$ 轮，现在剩余增广路长度均 $\geq |V|^{\frac{1}{2}}$ 。

这样的路径最多 $|V|^{\frac{1}{2}}$ 条，之后只需最多增广 $|V|^{\frac{1}{2}}$ 次。

综上所述，这样的网络 Dinic 算法时间复杂度为 $O(|E||V|^{\frac{1}{2}})$ 。

所以用最大流找二分图的最大匹配，时间复杂度为 $O(|E||V|^{\frac{1}{2}})$ 。

最小割

根据最大流最小割定理，我们可以先求出该网络的最大流。

如果需要求出最小割方案，可以从源点 S 搜索出能够到达哪些点。

此时所有可达点连向不可达点的边组成了最小割。

如果要最小化割边数量，可以将残余网络非满流边容量设为 $+\infty$ ，满流边容量设为 1，再求出最小割即可。（注意不要输出你建的反向边）

费用流

费用流在网络流的基础上，每条边多了单位容量的费用值 $w(u, v)$ 。

当 (u, v) 的流量为 $f(u, v)$ 时，其费用为 $f(u, v) \times w(u, v)$ 。

此时建 (u, v) 的反向边时，单位费用需要设为 $-w(u, v)$ 。

我们通常要解决的问题是，在找到该网络的最大流的同时，使得**总花费费用最小**，我们称找到的这样的最大流为**最小费用最大流**。

最小费用最大流算法大致流程

(该算法只解决无负圈的情况，存在负圈的做法将在之后讲解)

- 找到一条单位费用最小的增广路进行增广。
- 不断重复上操作直到不存在源点到汇点的路径。

该算法时间复杂度为 $O(|V||E||f|)$ ，其中 f 为该网络的最大流，为伪多项式复杂度，可构造网络卡到 $O(n^3 2^{\frac{n}{2}})$ 的时间复杂度。

但是在极大多数网络上，其效率仍然非常优秀。

下面对该算法正确性进行证明。

设当所有容量为 i 的流中，总费用最小的流费用为 f_i 。

由于没有负圈，则有 $f_0 = 0$ 。

若该算法得到的最大流费用不是最小的，我们考虑找到最小的 i ，使得当前流的容量为 i 时，在所有容量为 i 的流中**费用不是最小的**。

设当前总费用为 f'_i ，那么当前的最短增广路单位费用为 $f'_i - f_{i-1}$ 。

有 $f_i < f'_i$ ，则存在一条单位费用更小的增广路，显然其存在了**负圈**。

那么该负圈一定可以扔到得到 f_{i-1} 的流中使得费用更小，矛盾，由此得证该算法的正确性。

最小费用最大流算法实现

我们可以将最大流算法中 Dinic 或 EK 的 BFS 过程，改成最短路算法。

例如 Bellman-Ford 或者 SPFA 都可，时间复杂度为 $O(|V||E|)$ 。

剩下的部分几乎一致，不赘述。

上下界网络流

上下界网络流的每条边，存在一个下界容量 $b(u, v)$ 。

现在对于一个合法的流，其要保证每条边 (u, v) 的流量都满足 $b(u, v) \leq f(u, v) \leq c(u, v)$ 。

所以在上下界网络流时，**可能存在无解情况。**

无源汇上下界可行流

我们定义一个没有源点汇点的网络上的流，其只需要令每个点流入与流出的容量相等，并且每条边满足容量限制即可。

对于这样的网络存在上下界限制，若需要找出其中任意可行的流，可以先令每条边的流量为 $b(u, v)$ ，然后通过增加流量进行调整。

此时设某个点，流入容量减去流出容量的值为 D 。

我们建立超级源点 S' 与超级汇点 T' ，考虑构建一张新的网络。

考虑除了这两个点外，每个点可以分成三种情况：

- 该点 $D = 0$ ，已满足条件，不用管。
- 该点 $D > 0$ ，入流太多，需要出流抵消，则从 S' 向其连容量为 D 的边。
- 该点 $D < 0$ ，出流太多，则从其向 T' 连容量为 $-D$ 的边。

我们将边 (u, v) 在新的网络上的容量设为 $c(u, v) - b(u, v)$ ，发现在该网络上跑 S' 到 T' 的最大流，相当于增加流量调整的过程。

那么如果所有 S' 的出边与 T' 的入边满流，则调整出了可行流。

有源汇上下界可行流

此时网络已经有了源汇点 S, T 。

我们直接从 T 向 S 连容量为 $+\infty$ 的额外边，转化成无源汇的情况。

此时调整出的流，流量为额外边的流量。

有源汇上下界最大流

删去额外边再从 S 往 T 跑最大流，将残余网络上能流的再流完即可。

最大流流量为该次最大流流量加上之前额外边的流量。

有源汇上下界最小流

删去额外边从 T 往 S 跑最大流，将残余网络上能退流的再退完即可。

最小流流量为之前额外边的流量减去该次最大流流量。

有源汇上下界最小费用可行流/最大流

我们从无源汇的情况看起，做法是与无源汇可行流一致，只不过要记得将初始费用设成 $\sum_{(u,v) \in E} b(u,v) \times w(u,v)$ 。

然后建超级源点 S' 与超级汇点 T' ，同样的方法加新边（费用为 0），并且以同样的方法修改原有边的容量（费用不变）。

然后从 S' 向 T' 跑最小费用最大流，判断是否可行的方式也是一样的。

有源汇找可行流只需要从 T 向 S 建费用为 0，容量为 $+\infty$ 的边，最大流去掉额外边后，从 S 向 T 跑最小费用最大流。

总流量计算方式与之前一致，总费用在流过边时加上对应费用即可。



存在负圈的费用流

我们可以将所有负权边强制满流，相当于一条上下界皆为其容量的边。

此时要多一条反向边，其容量与原边相同，单位费用为原来的相反数，用来退流满流的原边。

那么可以直接转化成有源汇上下界最小费用最大流问题解决。

可以简单归纳证明，初始图不存在负圈，则流的过程中不会出现负圈。